STOCKWHISK SAAS PLATFORM

DOC-09

REST API Testing & Verification Specification

Endpoint Contracts, Request/Response Payloads, Status Codes & Auth

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
API Specification	API Testers, Backend Engineers, Integrators	v1.4.0 Production	✅ Verified & Approved

API Testing Specification

This document provides a comprehensive testing specification for the core API endpoints of the StockWhisk platform. It details expected behaviors, required payloads, and validation criteria, specifically emphasizing tenant data isolation and error handling.

General Testing Criteria for All Endpoints

For every endpoint tested, the following validations must be performed:
* **Authentication:** Verify that missing or invalid JWT tokens return a `401 Unauthorized`.
* **Tenant Isolation:** Verify that a user from Shop A cannot access, modify, or delete resources belonging to Shop B (returns `404 Not Found` or `403 Forbidden`).
* **Data Validation:** Verify that invalid request bodies (missing required fields, incorrect data types) return a `400 Bad Request` with descriptive error messages.
* **Method Allowed:** Verify that unsupported HTTP methods return a `405 Method Not Allowed`.

Authentication Endpoints

1. Initiate Registration

- **Method & URL:** `POST` `/api/accounts/register/initiate/`
- **Request Body:** `{"email": "newuser@test.com", "phone": "1234567890", ...}`
- **Expected Response:** `200 OK` (OTP sent)
- **Auth:** None
- **Errors:** `400` (Email/phone already exists)

2. Verify Registration

- **Method & URL:** `POST` `/api/accounts/register/verify/`
- **Request Body:** `{"email": "newuser@test.com", "otp": "123456"}`
- **Expected Response:** `201 Created` (Account created)
- **Auth:** None

- **Errors:** `400` (Invalid/Expired OTP)

3. Login

- **Method & URL:** `POST` `/api/accounts/login/`
- **Request Body:** `{"email": "user@test.com", "password": "password123"}`
- **Expected Response:** `200 OK` (Returns `access` and `refresh` JWTs)
- **Auth:** None
- **Errors:** `401` (Invalid credentials)

4. Refresh Token

- **Method & URL:** `POST` `/api/accounts/token/refresh/`
- **Request Body:** `{"refresh": "<refresh_token>"}`
- **Expected Response:** `200 OK` (Returns new `access` token)
- **Auth:** None
- **Errors:** `401` (Invalid/Expired refresh token)

5. Password Reset

- **Method & URL:** `POST` `/api/accounts/password-reset/`
- **Request Body:** `{"email": "user@test.com"}`
- **Expected Response:** `200 OK` (Reset link/OTP sent)
- **Auth:** None

Catalog Endpoints

1. Product Management

- **Method & URL:** `GET` `/api/catalog/products/` , `POST` `/api/catalog/products/`
- **Request Body (POST):** `{"name": "Item A", "price": 100, "category_id": 1, ...}`
- **Expected Response:** `200 OK` (List), `201 Created` (Details)
- **Auth:** Required (JWT)

2. Product Detail (CRUD)

- **Method & URL:** `GET` `/api/catalog/products/{id}/` , `PUT ...` , `DELETE ...`
- **Expected Response:** `200 OK` , `204 No Content` (DELETE)
- **Auth:** Required (JWT)
- **Errors:** `404` (Not found / Wrong tenant)

3. POS Product Grid (Optimized)

- **Method & URL:** `GET` `/api/catalog/products/?in_stock=1&light=1`
- **Expected Response:** `200 OK` (Optimized payload for POS UI)
- **Auth:** Required (JWT)

4. Product Units (Serialized Items)

- **Method & URL:** `GET` `/api/catalog/product-units/` , `POST ...`
- **Request Body (POST):** `{"product_id": 1, "serial_number": "SN12345"}`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

5. Categories & Brands

- **Method & URL:** `GET` `/api/catalog/categories/` , `POST ...` | `GET` `/api/catalog/brands/` , `POST ...`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

POS / Sales Endpoints

1. Checkout (Create Sale)

- **Method & URL:** `POST` `/api/pos/checkout/`
- **Request Body:** `{"items": [{"product_id": 1, "qty": 2}], "payment": {"amount": 200, "method": "CASH"}}`
- **Expected Response:** `201 Created` (Invoice details generated)
- **Auth:** Required (JWT)
- **Errors:** `400` (Insufficient stock, invalid payment amount)

2. Sales Invoices

- **Method & URL:** `GET` `/api/sales/sales/` (List), `GET` `/api/sales/sales/{id}/` (Detail)
- **Expected Response:** `200 OK`
- **Auth:** Required (JWT)

3. Add Payment to Invoice

- **Method & URL:** `POST` `/api/sales/sales/{id}/add_payment/`
- **Request Body:** `{"amount": 50, "method": "CARD"}`
- **Expected Response:** `200 OK` (Updated invoice status)
- **Auth:** Required (JWT)
- **Errors:** `400` (Payment exceeds due amount)

4. Correct Invoice

- **Method & URL:** `POST` `/api/sales/sales/{id}/correct/`
- **Request Body:** `{"reason": "Wrong item selected"}`
- **Expected Response:** `200 OK` (Status updated, inventory reversed)
- **Auth:** Required (Admin/Manager role)

5. Process Return

- **Method & URL:** `POST` `/api/sales/returns/`
- **Request Body:** `{"invoice_id": 1, "items": [{"product_id": 1, "qty_returned": 1}]}`
- **Expected Response:** `201 Created` (Return note created, inventory adjusted)
- **Auth:** Required (JWT)

Purchasing Endpoints

1. Purchase Orders

- **Method & URL:** `GET` `/api/purchasing/purchase-orders/`, `POST ...`
- **Request Body (POST):** `{"supplier_id": 1, "items": [{"product_id": 1, "qty": 10, "cost": 50}]}`
- **Expected Response:** `200 OK`, `201 Created`
- **Auth:** Required (JWT)

2. Receive Purchase Order

- **Method & URL:** `POST` `/api/purchasing/purchase-orders/{id}/receive/`
- **Request Body:** (Optional) Partial receipt details.
- **Expected Response:** `200 OK` (Inventory incremented, status updated)
- **Auth:** Required (JWT)
- **Errors:** `400` (Already received)

3. Suppliers

- **Method & URL:** `GET` `/api/purchasing/suppliers/`, `POST ...`
- **Expected Response:** `200 OK`, `201 Created`
- **Auth:** Required (JWT)

CRM Endpoints

1. Customers

- **Method & URL:** `GET` `/api/crm/customers/`, `POST ...`
- **Expected Response:** `200 OK`, `201 Created`
- **Auth:** Required (JWT)

2. Collect Due Payment

- **Method & URL:** `POST` `/api/crm/customers/{id}/pay-due/`
- **Request Body:** `{"amount": 100, "method": "BANK_TRANSFER"}`
- **Expected Response:** `200 OK` (Customer balance updated, ledger entry created)
- **Auth:** Required (JWT)

Accounting Endpoints

1. Expenses

- **Method & URL:** `GET /api/accounting/expenses/` , `POST ...`
- **Request Body (POST):** `{"category": "RENT", "amount": 1000}`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

2. Daily Settlements (Shifts)

- **Open Shift:** `POST /api/accounting/daily-settlements/open/`
  - **Body:** `{"opening_balance": 100}`
- **Close Shift:** `POST /api/accounting/daily-settlements/close/`
  - **Body:** `{"closing_balance_declared": 550}`
- **Expected Response:** `201 Created` , `200 OK`
- **Auth:** Required (JWT)

3. Financial Reports

- **Balance Sheet:** `GET /api/accounting/financial-position/`
- **P&L Report:** `GET /api/accounting/profit-report/`
- **Expected Response:** `200 OK` (Structured financial data)
- **Auth:** Required (Manager/Owner role)

Service Endpoints

1. Service Tickets

- **Method & URL:** `GET /api/service/tickets/` , `POST ...`
- **Request Body (POST):** `{"customer_id": 1, "device_info": "iPhone 12", "issue": "Broken screen"}`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

2. Ticket Status Transition

- **Method & URL:** `POST /api/service/tickets/{id}/change_status/`
- **Request Body:** `{"new_status": "IN_PROGRESS"}`
- **Expected Response:** `200 OK`
- **Auth:** Required (JWT)

3. Add Repair Part

- **Method & URL:** `POST /api/service/tickets/{id}/add_part/`
- **Request Body:** `{"product_id": 5, "qty": 1}`

- **Expected Response:** `200 OK` (Cost added to ticket, inventory adjusted)
- **Auth:** Required (JWT)

4. Warranties

- **Method & URL:** `GET` `/api/service/warranties/`
- **Expected Response:** `200 OK` (List of active/expired warranties)
- **Auth:** Required (JWT)